Trabajo Práctico N° 5

Compresión de datos

Compresión de datos. Métodos de compresión. Métodos de compresión sin pérdida. Algoritmo de Huffman. Algoritmo de Shannon-Fano. Run Length Coding (RLC). Métodos de compresión con pérdida.

1. Dadas las siguientes fuentes, generar códigos de Huffman y de Shannon-Fano:

Símbolos	S1	S2	S3	S4
Fuente A	0.2	0.2	0.3	0.3
Fuente B	0.4	0.25	0.25	0.1

2. Construir códigos de Huffman y de Shannon-Fano para fuentes de información que emiten los siguientes mensajes representativos:
    a. ABCDABCBDCBAAABBBCBCBABADBCBABCBDBCCCAAABB
    b. AOEAOEOOOOEOAOEOOEOOEOAOAOEOEUUUIEOEOEO
3. Desarrollar dos funciones en Python que reciban como parámetro una lista con la distribución de probabilidades de una fuente de información y generen una lista de cadenas de caracteres con codificaciones binarias de Huffman y de Shannon-Fano.
4. Para una fuente de información con distribución de probabilidades:
   P = { 0.385, 0.154, 0.128, 0.154, 0.179 }
    a. Calcular la entropía de la fuente
    b. Generar una codificación de Huffman
    c. Obtener una codificación de Shannon-Fano
    d. Graficar los árboles binarios que surgen de cada codificación
    e. Comparar la longitud media, el rendimiento y la redundancia de cada código
5. Dada una fuente de información que emite el siguiente mensaje representativo:
   587847845253686698957451236562536989896564521217023002236 59
    a. Calcular la entropía de la fuente
    b. Construir una codificación de Huffman
    c. Generar una codificación de Shannon-Fano
    d. Comparar la longitud media, el rendimiento y la redundancia de cada código
6. De acuerdo a la codificación del ejercicio 2 del TP4, decodificar los siguientes mensajes:
    a. ABACBAACABABAACBABA
    b. BACBAAABAAACBABACAB
    c. CBAABACBABAAACABABA

7. Implementar funciones en Python que reciban como parámetros: una cadena de caracteres que contenga un alfabeto fuente y una lista de cadenas de caracteres que almacena una codificación en el alfabeto binario, y resuelvan lo siguiente:
    a. Dada una cadena de caracteres con un mensaje escrito en el alfabeto fuente, devolver una secuencia de bytes (bytearray) que contenga el mensaje codificado.
    b. Dada una secuencia de bytes, decodificar y retornar el mensaje original.

    Sugerencia: manipular el mensaje codificado como una cadena de caracteres de unos y ceros, tanto para codificar como para decodificar, y realizar las conversiones entre binarios y enteros con las funciones de casteo correspondientes.
8. Codificar una función en Python que reciba como parámetros: una cadena de caracteres con un mensaje y una secuencia de bytes (bytearray) con ese mensaje codificado y calcule la tasa de compresión.
9. Dada la siguiente tabla de probabilidades:

Símbolo	Prob	Símbolo	Prob	Símbolo	Prob	Símbolo	Prob
espacio	0.175990	D	0.038747	L	0.048149	S	0.058406
,	0.014093	E	0.101604	M	0.021041	T	0.031093
.	0.015034	F	0.004873	N	0.050490	U	0.033240
:	0.000542	G	0.008762	Ñ	0.002018	V	0.008930
;	0.002109	H	0.007953	O	0.073793	W	0.000012
A	0.111066	I	0.049740	P	0.019583	X	0.000706
B	0.015368	J	0.003706	Q	0.010246	Y	0.007851
C	0.030176	K	0.000034	R	0.051446	Z	0.003199

    Utilizando las funciones desarrolladas en los ejercicios 3 y 7, comprimir un mensaje mediante el algoritmo de Huffman y/o Shannon-Fano y almacenarlo en un archivo binario. Enviar el archivo a un compañero, quien tendrá que extraer y descomprimir el mensaje. Ambos deben calcular la tasa de compresión, el rendimiento y la redundancia.
10. Comprimir los siguientes mensajes utilizando el algoritmo RLC:
    a. XXXYZZZZ
    b. AAAABBBCCDAA
    c. UUOOOOAAAIEUUUU
11. Realizar una función en Python que reciba como parámetro una cadena de caracteres con un mensaje y devuelva una secuencia de bytes (bytearray) que contenga el mensaje comprimido con RLC, utilizando un byte para almacenar la representación en código ASCII del carácter y otro byte para el número.
12. Volver a comprimir los mensajes del ejercicio 10 (utilizando la función desarrollada) y calcular la tasa de compresión de cada uno.

Resultados:

1. $L_A = \{ 2, 2, 2, 2 \}$ $L_B = \{ 1, 3, 2, 3 \}$

2. a. $S = \{ A, B, C, D \}$ $P = \{ 0.26, 0.4, 0.24, 0.1 \}$ $L = \{2, 1, 3, 3\}$

   b. $S = \{ A, E, I, O, U \}$ $P = \{ 0.13, 0.28, 0.02, 0.49, 0.08 \}$ $L = \{3, 2, 4, 1, 4\}$

4. a. $H_2(S) = 2.1854$ bits

   b. $L_{Huffman} = \{ 1, 3, 3, 3, 3 \}$

   c. $L_{Shannon\text{-}Fano} = \{ 2, 3, 3, 2, 2 \}$

   e. $L_{Huffman} = 2.2300$ bits $\eta_{Huffman} = 0.9800$ $R_{Huffman} = 0.0200$

   $L_{Shannon\text{-}Fano} = 2.2820$ bits $\eta_{Shannon\text{-}Fano} = 0.9576$ $R_{Shannon\text{-}Fano} = 0.0424$

5. a. $H_2(S) = 3.2041$ bits

   b. $L_{Huffman} = \{ 4, 4, 3, 3, 4, 3, 3, 4, 3, 3 \}$

   c. $L_{Shannon\text{-}Fano} = \{ 4, 4, 3, 4, 4, 2, 3, 4, 3, 4 \}$

   d. $L_{Huffman} = 3.2373$ bits $\eta_{Huffman} = 0.9898$ $R_{Huffman} = 0.0102$

   $L_{Shannon\text{-}Fano} = 3.2542$ bits $\eta_{Shannon\text{-}Fano} = 0.9846$ $R_{Shannon\text{-}Fano} = 0.0154$

6. a. S3 S1 S4 S3 S2 S3 S1 S3 S4 S1

   b. S1 S4 S3 S3 S1 S3 S3 S4 S1 S2

   c. S4 S3 S1 S4 S1 S3 S3 S2 S3 S1

10. a. X3Y1Z4

    b. A4B3C2D1A2

    c. U2O4A3I1E1U4

12. a. 4 / 3

    b. 6 / 5

    c. 5 / 4